MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: Gendal Lal

Subject & Subject Code: Theory of Computation & 4CSE4-03, 4AM4-03, 4AD4-03

Branch: CSE, AIML and AI & DS Batch:

Semester: IV **Session:** 2023-24(Even Sem.)

Unit No.: 5 **Date of Submission: 11/03/2024**

Lect.No.	Topics Covered	Page No.	Total pages
1	PDA definition P,NP,NP complete	1-7	7
2	PDA acceptability NP hard problem	8-12	5
3	NPDA un-decidability,examples	13-14	2
4	PDA and CFG vertex cover problem	15-16	2
5	CFG to PDA hamiltonian path problems	17-18	2
6	Pumping lemma for CFL ,Traveling sales man	19-21	3

References Book:

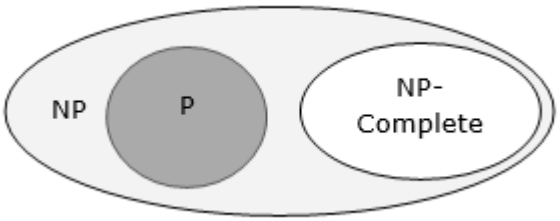
1. "Introduction to the Theory of Computation "by Micheal Sipser
2. "Introduction to computability "by Fred C Henie

FACULTY **HOD** **DEAN ACADEMICS**

Lecture-01

P, NP, NP Complete & NP Hard Problem

A problem is in the class NPC if it is in NP and is as **hard** as any problem in NP. A problem is **NP-hard** if all problems in NP are polynomial time reducible to it, even though it may not be in NP itself.



If a polynomial time algorithm exists for any of these problems, all problems in NP would be polynomial time solvable. These problems are called **NP-complete**. The phenomenon of NP-completeness is important for both theoretical and practical reasons.

Definition of NP-Completeness

A language **B** is **NP-complete** if it satisfies two conditions

- **B** is in NP
- Every **A** in NP is polynomial time reducible to **B**.

If a language satisfies the second property, but not necessarily the first one, the language **B** is known as **NP-Hard**. Informally, a search problem **B** is **NP-Hard** if there exists some **NP-Complete** problem **A** that Turing reduces to **B**.

MTRC

The problem in NP-Hard cannot be solved in polynomial time, until P = NP. If a problem is proved to be NPC, there is no need to waste time on trying to find an efficient algorithm for it. Instead, we can focus on design approximation algorithm.

NP-Complete Problems

Following are some NP-Complete problems, for which no polynomial time algorithm is known.

- •Determining whether a graph has a Hamiltonian cycle
- •Determining whether a Boolean formula is satisfiable, etc.

NP-Hard Problems

The following problems are NP-Hard

- •The circuit-satisfiability problem
- •Set Cover
- •Vertex Cover
- •Travelling Salesman Problem

In this context, now we will discuss TSP is NP-Complete

TSP is NP-Complete

The traveling salesman problem consists of a salesman and a set of cities. The salesman has to visit each one of the cities starting from a certain one and returning to the same city. The challenge of the problem is that the traveling salesman wants to minimize the total length of the trip

Proof

4	Universal Turning Machine	26-27	2
5	Multi-Track turning Machines	28-28	1
6	Properties of RL & REL Languages	29-31	3

References Book:

1. "Introduction to the Theory of Computation "by Micheal Sipser
2. "Introduction to computability "by Fred C Henie

FACULTY **HOD** **DEAN ACADEMICS**

MTRC

$E' = \{(i,j) : i,j \in V \text{ and } i \neq j$

Thus, the cost function is defined as follows −

$t(i,j) = \{0 \text{ if } (i,j) \in E \text{ otherwise}$

Now, suppose that a Hamiltonian cycle h exists in G. It is clear that the cost of each edge in h is 0 in G' as each edge belongs to E. Therefore, h has a cost of 0 in G'. Thus, if graph G has a Hamiltonian cycle, then graph G' has a tour of 0 cost.

Conversely, we assume that G' has a tour h' of cost at most 0. The cost of edges in E' are 0 and 1 by definition. Hence, each edge must have a cost of 0 as the cost of h' is 0. We therefore conclude that h' contains only edges in E.

We have thus proven that G has a Hamiltonian cycle, if and only if G' has a tour of cost at most 0. TSP is NP-complete.

Lecture-02

Un-Decidability

Identifying languages (or problems*) as decidable, undecidable or partially decidable is a very common question in GATE. With correct knowledge and ample experience, this question becomes very easy to solve.

Lets start with some definitions:-

Decidable language -A decision problem P is said to be decidable (i.e., have an algorithm) if the language L of all yes instances to P is decidable.
Example- (I) (Acceptance problem for DFA) Given a DFA does it accept a given word?
(II) (Emptiness problem for DFA) Given a DFA does it accept any word?

(III) (Equivalence problem for DFA) Given two DFAs, do they accept the same language?

Undecidable language -- A decision problem P is said to be undecidable if the language L of all yes instances to P is not decidable or a language is undecidable if it is not decidable. An undecidable language maybe a partially decidable language or something else but not decidable. If a language is not even partially decidable , then there exists no Turing machine for that language.

Partially decidable or Semi-Decidable Language -- A

MITRC

decision problem P is said to be semi-decidable (i.e., have a semi-algorithm) if the language L of all yes instances to P is RE. A language 'L' is partially decidable if 'L' is a RE but not REC language.

Recursive language(REC) – A language 'L' is said to be recursive if there exists a Turing machine which will accept all the strings in 'L' and reject all the strings not in 'L'. The Turing machine will halt every time and give an answer(accepted or rejected) for each and every string input. A language 'L' is decidable if it is a recursive language. All decidable languages are recursive languages and vice-versa.

Recursively enumerable language(RE) – A language 'L' is said to be a recursively enumerable language if there exists a Turing machine which will accept (and therefore halt) for all the input strings which are in 'L' but may or may not halt for all input strings which are not in 'L'. By definition , all REC languages are also RE languages but not all RE languages are REC languages.

MTRC

Lecture-03

Vertex Cover Problem

A vertex-cover of an undirected graph G = (V, E) is a subset of vertices V' ⊆ V such that if edge (u, v) is an edge of G, then either u in V or v in V' or both.

Find a vertex-cover of maximum size in a given undirected graph. This optimal vertexcover is the optimization version of an NP-complete problem. However, it is not too hard to find a vertex-cover that is near optimal.

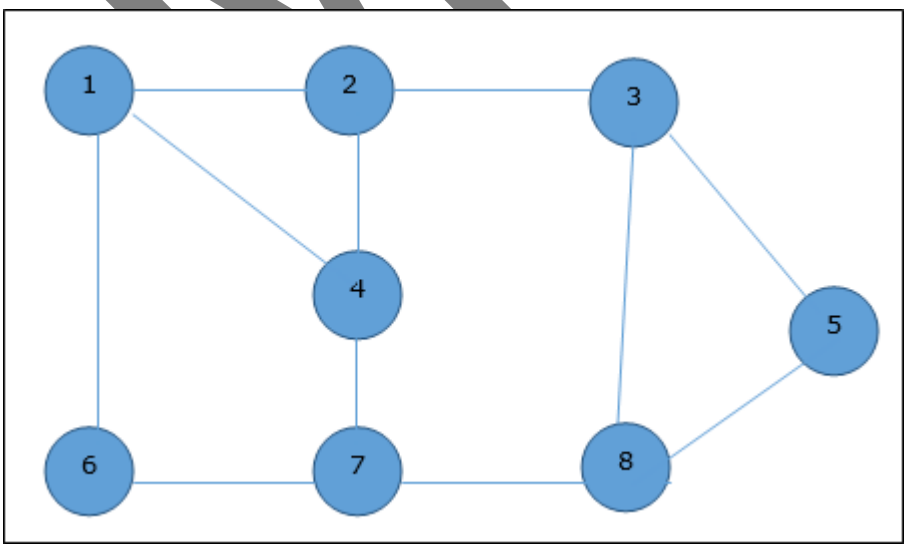
APPROX-VERTEX_COVER (G: Graph) c ← { } E' ← E[G]

```
while E' is not empty do
   Let (u, v) be an arbitrary edge of E' c ← c U {u, v}
   Remove from E' every edge incident on either u or v
return c
```

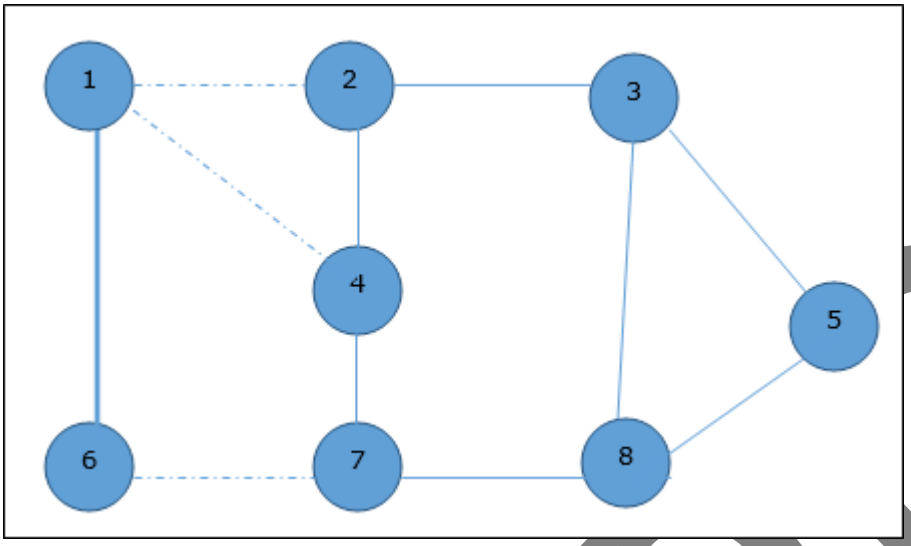
Example

The set of edges of the given graph is –

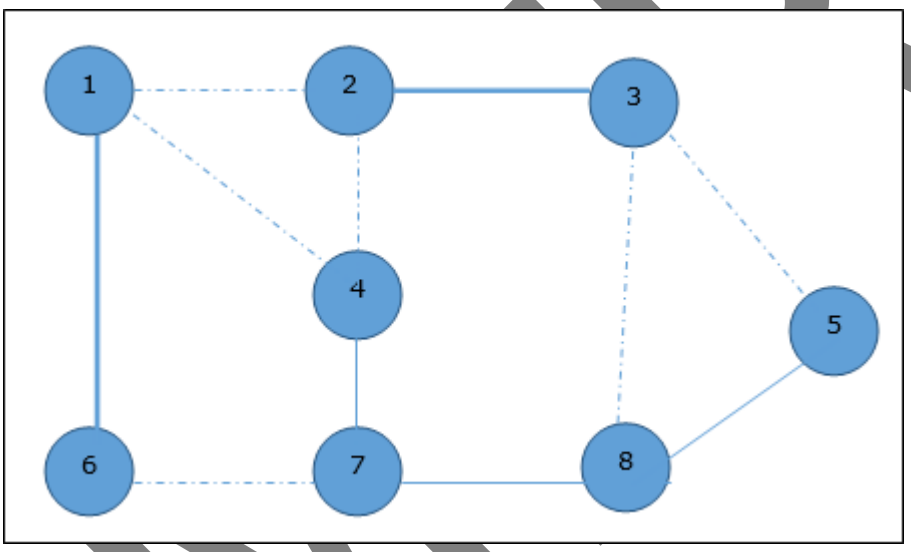
{(1,6),(1,2),(1,4),(2,3),(2,4),(6,7),(4,7),(7,8),(3,8),(3,5),(8,5)}



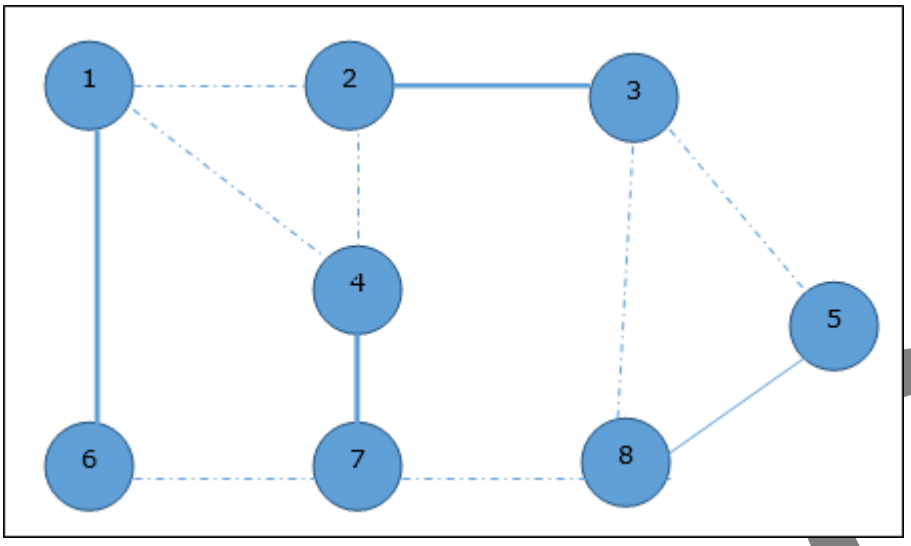
Now, we start by selecting an arbitrary edge (1,6). We eliminate all the edges, which are either incident to vertex 1 or 6 and we add edge (1,6) to cover.



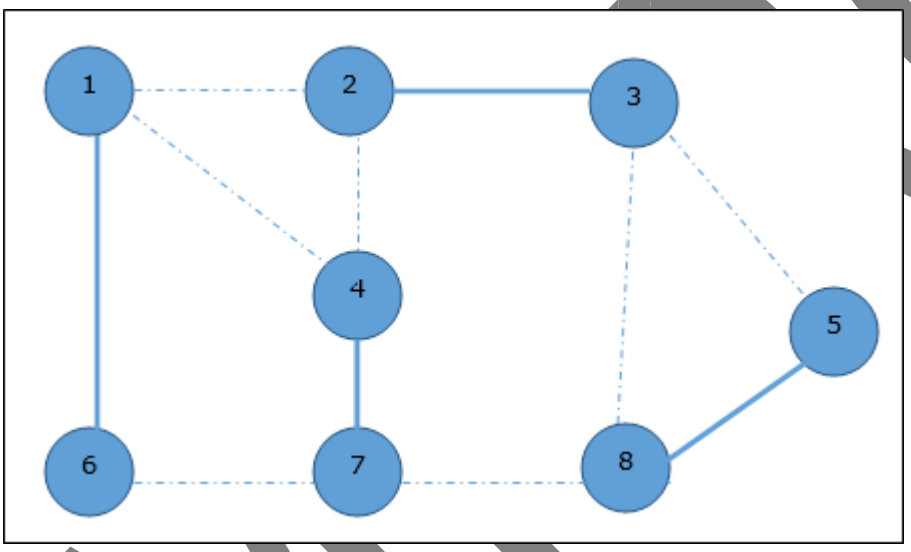
In the next step, we have chosen another edge (2,3) at random



Now we select another edge (4,7).



We select another edge (8,5).



Hence, the vertex cover of this graph is {1,2,4,5}.

Lecture-04

Hamiltonian Cycle Problem

In an undirected graph, the Hamiltonian path is a path, that visits each vertex exactly once, and the Hamiltonian cycle or circuit is a Hamiltonian path, that there is an edge from the last vertex to the first vertex.

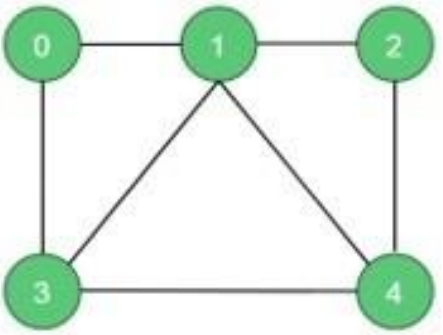
In this problem, we will try to determine whether a graph contains a Hamiltonian cycle or not. And when a Hamiltonian cycle is present, also print the cycle.

Input and Output

Input:

The adjacency matrix of a graph G(V, E).

0	1	0	1	0
1	0	1	1	1
0	1	0	0	1
1	1	0	0	1
0	1	1	1	0



Output:

The algorithm finds the Hamiltonian path of the given graph. For this case it is (0, 1, 2, 4, 3, 0). This graph has some other Hamiltonian paths.

If one graph has no Hamiltonian path, the algorithm should return false.

Algorithm

isValid(v, k)

Input – Vertex v and position k.

Output – Checks whether placing v in the position k is valid or not.

```
Begin
  if there is no edge between node(k-1) to v, then
    return false
  if v is already taken, then
    return false
  return true; //otherwise it is valid
End
```

cycleFound(node k)

Input – node of the graph.

Output – True when there is a Hamiltonian Cycle, otherwise false.

```
Begin
  if all nodes are included, then
    if there is an edge between nodes k and 0, then
      return true
    else
      return false;

  for all vertex v except starting point, do
    if isValid(v, k), then //when v is a valid edge
      add v into the path
      if cycleFound(k+1) is true, then
        return true
```

```
        otherwise  remove v from  the path
   done
   return  false
End
```

MITRC